



Université Sultan Moulay Slimane
Faculté Polydisciplinaire de Béni Mellal
Département des technologies nouvelles



PROJET DE FIN D'ÉTUDES

Présenté pour l'obtention du diplôme de
Master en Data Science et Sécurité des Systèmes d'Information

**TITRE : Mise en place d'une plateforme de FabLab
virtuel intelligent en Industrie 4.0**

Présenté par :

M. ESSAID ABDELKHALEK

Sous la direction de :

Pr. Soufiane BELHOUIDEG

**Organisme d'accueil : ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE
BÉNI MELLAL**

Cadre : Projet de Fin d'Études – Virtual FabLab

Soutenu le : À compléter

Devant le Jury :

Pr. Nom Prénom FP Béni Mellal

Pr. Nom Prénom FP Béni Mellal

Pr. Nom Prénom FP Béni Mellal

REMERCIEMENTS

Je tiens à exprimer mes sincères remerciements à mon encadrant pédagogique pour son accompagnement, ses conseils et son suivi tout au long de la réalisation de ce projet de fin d'études.

Je remercie également l'ensemble du corps professoral et administratif de l'ENSA Béni Mellal et de l'Université Sultan Moulay Slimane pour la qualité de la formation reçue durant mon parcours académique.

Enfin, j'adresse mes remerciements à ma famille et à toutes les personnes qui m'ont soutenu directement ou indirectement durant la réalisation de ce travail.

RÉSUMÉ

Ce projet de fin d'études porte sur la conception et le développement d'une plateforme virtuelle de type FabLab intégrant la simulation 3D, l'Internet des Objets et la Data Science. La solution proposée permet de visualiser et de superviser des machines telles qu'une imprimante 3D et une machine CNC dans un environnement interactif.

La plateforme repose sur une architecture moderne combinant une interface web, un backend applicatif, une simulation 3D, une télémétrie simulée via MQTT et un module d'analyse de données. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

Mots-clés : FabLab, Industrie 4.0, Simulation 3D, IoT, MQTT, Data Science, Maintenance prédictive, CNC, Imprimante 3D.

ABSTRACT

This final year project focuses on the design and development of a virtual FabLab platform integrating 3D simulation, Internet of Things technologies and Data Science. The proposed solution enables the visualization and supervision of machines such as a 3D printer and a CNC machine within an interactive environment.

The platform is based on a modern architecture combining a web interface, an application backend, a 3D simulation module, MQTT-simulated telemetry and a data analysis component. In this version, telemetry data is simulated to validate the architecture, MQTT flows, monitoring and analytics module. The platform remains designed for later connection to real sensors.

Keywords: FabLab, Industry 4.0, 3D Simulation, IoT, MQTT, Data Science, Predictive Maintenance, CNC, 3D Printer.

Table des matières

Remerciements	2
Résumé	4
Abstract	6
Liste des figures	10
Liste des tableaux	11
Liste des abréviations	13
Introduction générale	15
1 Présentation de l'organisme d'accueil et cadre général du projet	16
1.1 Introduction	17
1.2 Présentation de l'ENSA Béni Mellal	17
1.3 Missions et formations	17
1.3.1 Missions de l'établissement	17
1.3.2 Formations proposées	18
1.4 Organigramme de l'établissement	18
1.5 Contexte du stage	19
1.6 Problématique	20
1.7 Objectifs du projet	20
1.8 Présentation générale du Virtual FabLab	21
1.9 Conclusion	22
2 Analyse des besoins et conception	23
2.1 Introduction	24
2.2 Étude des besoins	24

2.3	Besoins fonctionnels	24
2.3.1	Besoins de l'étudiant	24
2.3.2	Besoins de l'administrateur	25
2.4	Besoins non fonctionnels	25
2.5	Acteurs du système	26
2.6	Cas d'utilisation	26
2.7	Architecture générale	27
2.8	Description des modules	28
2.9	Diagramme de classes	29
2.10	Conception du système	30
2.10.1	Conception de l'authentification	30
2.10.2	Conception de la gestion des machines	31
2.10.3	Conception de la réservation	31
2.10.4	Conception de la télémétrie	31
2.10.5	Conception de l'analyse intelligente	31
2.11	Technologies utilisées	31
2.11.1	Next.js	32
2.11.2	FastAPI	32
2.11.3	Three.js	32
2.11.4	Blender	32
2.11.5	MQTT	32
2.11.6	Python	33
2.11.7	Tailwind CSS	33
2.11.8	Scikit-learn	33
2.11.9	Visual Studio Code	33
2.11.10	Technologies et équipements IoT	33
2.12	Conclusion	35
3	Réalisation et implémentation	36
3.1	Introduction	37
3.2	Architecture frontend/backend	37
3.3	Organisation des dossiers	38
3.4	Implémentation du backend	38

3.4.1	API FastAPI	38
3.4.2	Base de données	39
3.5	Système d’authentification	39
3.6	Gestion des utilisateurs et de l’administration	40
3.7	Gestion des machines	40
3.8	Système de réservation	41
3.9	FabLab virtuel en 3D	42
3.10	Simulation CNC	42
3.11	Simulation de l’imprimante 3D	43
3.12	Interprétation des fichiers G-code et NC	44
3.13	Télémétrie simulée avec MQTT	44
3.14	Tableau de bord Data Science	45
3.15	Alertes d’anomalies et score de santé	46
3.16	Graphiques et historique de télémétrie	47
3.17	Communication entre frontend et backend	47
3.18	Interfaces réalisées	48
3.19	Conclusion	48
4	Validation et résultats	49
4.1	Introduction	50
4.2	Validation fonctionnelle de la plateforme	50
4.2.1	Tests du frontend	50
4.2.2	Tests du backend FastAPI	51
4.2.3	Communication frontend/backend	51
4.2.4	Évaluation fonctionnelle du module Data Science	52
4.3	Résultats obtenus	53
4.4	Limites actuelles et perspectives	53
4.4.1	Limites des données et validation	53
4.4.2	Perspectives d’évolution	54
4.5	Conclusion	54
	Conclusion générale	57
	Bibliographie et webographie	58

Table des figures

1.1	Organigramme de l'ENSA Béni Mellal	19
1.2	Vue générale prévue de la plateforme Virtual FabLab	22
2.1	Diagramme de cas d'utilisation de la plateforme Virtual FabLab	27
2.2	Architecture générale de la plateforme Virtual FabLab	28
2.3	Diagramme de classes de la plateforme Virtual FabLab	30
3.1	Architecture d'implémentation frontend/backend	37
3.2	Schéma logique de la base de données	39
3.3	Interface de gestion des utilisateurs	40
3.4	Interface de gestion des machines	41
3.5	Interface de réservation des machines	41
3.6	Vue 3D du FabLab virtuel	42
3.7	Simulation d'une trajectoire CNC	43
3.8	Simulation d'un fichier G-code pour imprimante 3D	43
3.9	Processus d'interprétation et de validation des fichiers G-code/NC	44
3.10	Flux MQTT implémenté dans la plateforme	45
3.11	Tableau de bord administrateur et monitoring intelligent	46
3.12	Cartes de prédiction et score de santé des machines	47
3.13	Historique des mesures d'une machine	47
3.14	Communication entre le frontend et le backend	48
4.1	Schéma de communication entre l'interface Next.js et le backend FastAPI	52

Liste des tableaux

3.1	Organisation générale du projet	38
-----	---	----

LISTE DES ABRÉVIATIONS

PFE Projet de Fin d'Études

D3SI Data Science et Sécurité des Systèmes d'Information

ENSA École Nationale des Sciences Appliquées

FPBM Faculté Polydisciplinaire de Béni Mellal

IoT Internet des Objets

MQTT Message Queuing Telemetry Transport

CNC Commande Numérique par Calculateur

IA Intelligence Artificielle

API Application Programming Interface

JWT JSON Web Token

JSON JavaScript Object Notation

REST Representational State Transfer

UI User Interface

UX User Experience

INTRODUCTION GÉNÉRALE

Les avancées technologiques liées à l'Industrie 4.0 transforment progressivement les environnements de fabrication, d'apprentissage et de prototypage. Les FabLabs représentent aujourd'hui des espaces essentiels pour l'innovation, car ils permettent aux étudiants, chercheurs et porteurs de projets d'accéder à des machines de fabrication numérique telles que les imprimantes 3D et les machines CNC.

Cependant, l'utilisation classique d'un FabLab reste généralement limitée par la présence physique, la disponibilité des machines et l'absence d'outils numériques avancés pour la supervision et l'analyse des données. Dans ce contexte, la mise en place d'une plateforme virtuelle devient une solution pertinente pour améliorer l'accessibilité, la visualisation, la gestion et l'exploitation intelligente des équipements.

Le présent projet vise à concevoir et développer une plateforme virtuelle FabLab intégrant une interface web moderne, une simulation 3D interactive, une communication IoT simulée via MQTT et un module de Data Science destiné à l'analyse des données machines et à la maintenance prédictive. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

Ce rapport est organisé comme suit. Le premier chapitre présente l'organisme d'accueil ainsi que le cadre général du projet, la problématique et les objectifs visés. Le deuxième chapitre est consacré à l'analyse des besoins, à la conception du système et aux technologies utilisées. Le troisième chapitre détaille la réalisation et l'implémentation effective de la plateforme. Enfin, le quatrième chapitre présente la validation fonctionnelle, les résultats obtenus, les limites actuelles et les perspectives d'évolution de la solution.

CHAPITRE 1

PRÉSENTATION DE L'ORGANISME D'ACCUEIL ET CADRE GÉNÉRAL DU PROJET

1.1 Introduction

Le Projet de Fin d'Études constitue une étape importante dans le parcours du Master en Data Science et Sécurité des Systèmes d'Information. Il permet de mobiliser les connaissances acquises durant la formation autour d'un projet concret, combinant analyse, conception, développement logiciel et validation technique.

Le présent projet, intitulé **Virtual FabLab**, s'inscrit dans le contexte de la transformation numérique des environnements de fabrication et d'apprentissage. Il vise à proposer une plateforme web permettant de visualiser un FabLab virtuel, de gérer ses machines, de simuler des opérations de fabrication numérique et d'exploiter des données IoT simulées pour la supervision intelligente.

Ce chapitre présente l'organisme d'accueil, le cadre général du projet, la problématique traitée ainsi que les objectifs poursuivis.

1.2 Présentation de l'ENSA Béni Mellal

L'École Nationale des Sciences Appliquées de Béni Mellal, désignée dans ce rapport par ENSA BM, est un établissement public d'enseignement supérieur relevant de l'Université Sultan Moulay Slimane. Elle participe à la formation de profils scientifiques et technologiques capables de répondre aux besoins du marché dans les domaines de l'ingénierie, de l'informatique, de la data science, des réseaux et des systèmes d'information.

L'ENSA BM offre un environnement académique favorable à l'apprentissage par projet, à l'innovation et à l'expérimentation technique. Dans ce cadre, les projets de fin d'études occupent une place essentielle, car ils permettent aux étudiants de confronter leurs compétences à des problématiques réelles ou proches du réel.

1.3 Missions et formations

1.3.1 Missions de l'établissement

Les missions principales de l'ENSA BM peuvent être résumées comme suit :

- assurer une formation scientifique et technique de haut niveau ;
- développer les compétences en ingénierie, en informatique et en technologies numériques ;
- encourager la recherche appliquée, l'innovation et l'esprit d'initiative ;

- préparer les étudiants à l'intégration professionnelle et à la conduite de projets complexes ;
- contribuer au développement régional et national à travers la formation de profils qualifiés.

1.3.2 Formations proposées

L'École Nationale des Sciences Appliquées de Béni Mellal propose des formations d'ingénieurs orientées vers les sciences appliquées, les technologies et l'ingénierie. Ces formations visent à développer des compétences scientifiques et techniques adaptées aux besoins du monde professionnel.

Les domaines de formation couvrent notamment :

- l'informatique et les technologies numériques ;
- les systèmes embarqués et connectés ;
- les télécommunications ;
- les systèmes industriels ;
- l'ingénierie et les sciences appliquées.

Dans ce contexte technologique, l'ENSA de Béni Mellal constitue un environnement adapté à la réalisation du projet Virtual FabLab, qui intègre le développement web, la simulation 3D, les technologies IoT et l'exploitation des données.

1.4 Organigramme de l'établissement

L'organisation de l'ENSA BM repose sur une structure administrative et pédagogique permettant d'assurer le suivi des formations, l'encadrement des étudiants et la gestion des activités académiques.

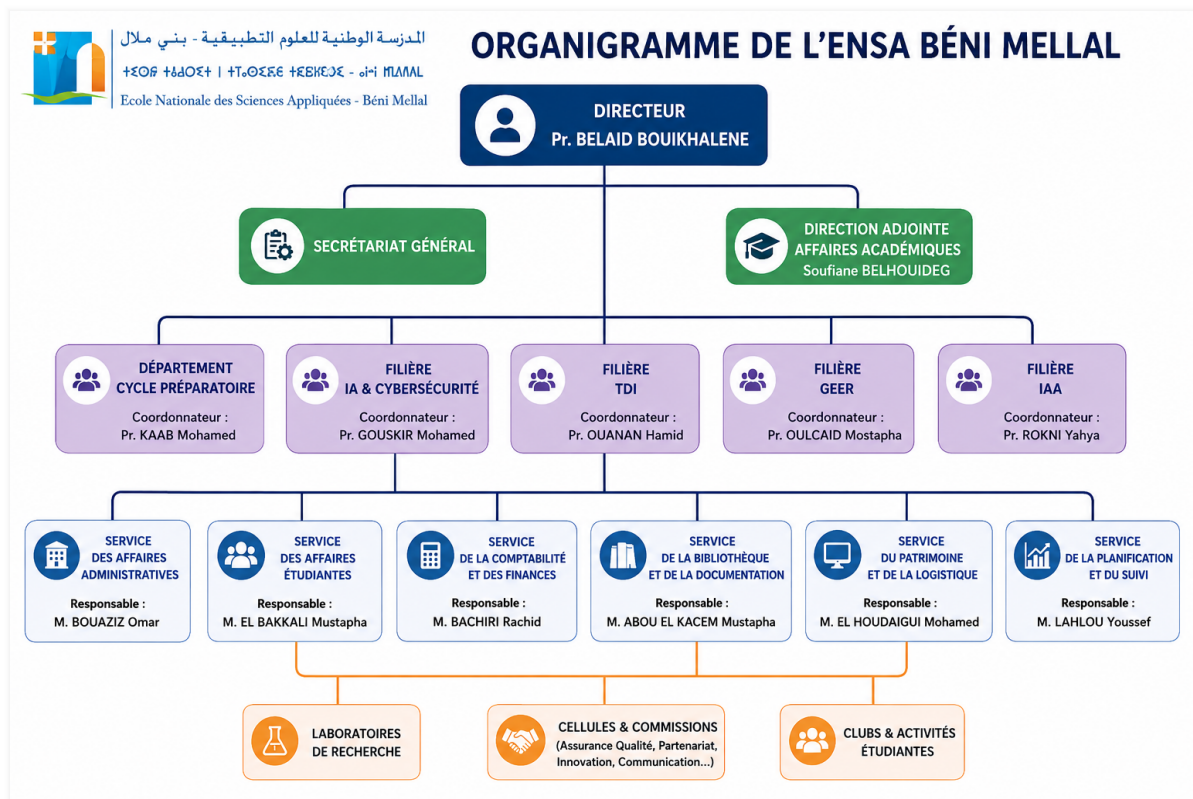


Figure 1.1: Organigramme de l'ENSA Béni Mellal

La figure 1.1 présente l'organisation générale de l'établissement. Cette structure permet de situer le projet dans son environnement académique et de comprendre le cadre institutionnel dans lequel il a été réalisé.

1.5 Contexte du stage

Le projet a été réalisé dans un contexte académique orienté vers l'innovation numérique et l'Industrie 4.0. Les FabLabs constituent aujourd'hui des espaces importants pour l'apprentissage pratique, le prototypage rapide et la fabrication numérique. Ils mettent à disposition des machines telles que les imprimantes 3D, les machines CNC ou les découpeuses laser.

Cependant, l'accès à ces équipements peut être limité par plusieurs contraintes : disponibilité des machines, besoin de présence physique, difficulté de supervision à distance et absence d'outils unifiés permettant de relier gestion, simulation et analyse de données.

Dans ce contexte, la création d'un FabLab virtuel représente une réponse pertinente. Elle permet de préparer les utilisateurs avant l'utilisation réelle des machines, de simuler des trajectoires de fabrication et d'introduire une couche de supervision intelligente fondée sur les données. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être

connectée ultérieurement à des capteurs réels.

1.6 Problématique

Les FabLabs traditionnels reposent principalement sur une interaction physique avec les équipements. Cette approche est utile pour l'apprentissage pratique, mais elle présente certaines limites dans un contexte pédagogique moderne. Les étudiants ne disposent pas toujours d'un moyen simple pour visualiser les machines, tester un fichier de fabrication ou consulter l'état des équipements avant une séance.

Par ailleurs, les machines de fabrication numérique produisent ou pourraient produire des données de fonctionnement telles que la température, la vibration, la durée d'utilisation, la vitesse du moteur ou les erreurs signalées. Ces données sont précieuses pour la supervision et la maintenance, mais elles restent souvent peu exploitées dans un cadre académique. Dans la version réalisée, ces mesures ne proviennent pas encore de capteurs physiques : elles sont générées par un simulateur MQTT afin de tester les traitements et les interfaces dans des conditions contrôlées.

La problématique du projet peut donc être formulée ainsi :

Comment concevoir et réaliser une plateforme virtuelle permettant de gérer un FabLab, de simuler ses machines et d'exploiter des données IoT simulées pour la supervision intelligente et la maintenance prédictive ?

1.7 Objectifs du projet

L'objectif principal du projet est de concevoir et développer une plateforme web nommée **Virtual FabLab**, intégrant la gestion des utilisateurs et des machines, la visualisation 3D, la simulation de fichiers de fabrication et l'exploitation de données IoT simulées.

Les objectifs spécifiques sont les suivants :

- mettre en place une interface web moderne destinée aux étudiants et aux administrateurs ;
- permettre l'inscription, l'authentification et la gestion des profils utilisateurs ;
- gérer les machines du FabLab, leurs types, leurs états et leur positionnement dans l'espace 3D ;
- offrir un système de réservation des machines avec validation administrative ;
- construire une scène 3D interactive représentant le FabLab et ses équipements ;

- simuler le comportement d'une imprimante 3D et d'une machine CNC à partir de fichiers G-code ou NC ;
- intégrer une télémétrie simulée transmise via MQTT ;
- analyser les données simulées afin de détecter les anomalies et d'estimer l'état de santé des machines ;
- afficher les indicateurs de supervision dans un tableau de bord administrateur.

1.8 Présentation générale du Virtual FabLab

La plateforme Virtual FabLab est conçue comme une application web complète reposant sur une architecture frontend/backend. Le frontend assure l'expérience utilisateur, la visualisation du laboratoire virtuel, les interfaces de réservation, les écrans administrateur et les vues de simulation. Le backend fournit les API, la persistance des données, l'authentification, la gestion des rôles, la réception des données MQTT et les services d'analyse.

La solution réalisée couvre plusieurs modules complémentaires :

- un module d'authentification basé sur des jetons JWT ;
- un module de gestion des utilisateurs et des droits d'accès ;
- un module de gestion des machines et de leurs emplacements dans le FabLab virtuel ;
- un module de réservation avec suivi des demandes ;
- un module de visualisation 3D du laboratoire ;
- un module de simulation G-code pour imprimante 3D et CNC ;
- un module MQTT permettant l'ingestion de télémétrie simulée ;
- un module Data Science pour la détection d'anomalies, le score de santé, l'estimation indicative du risque de panne et les recommandations de maintenance.

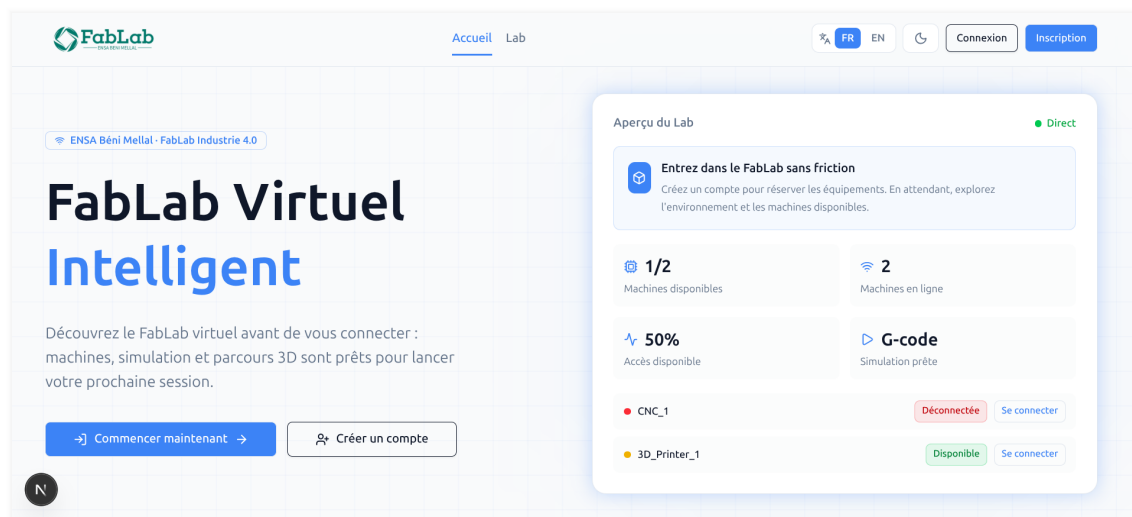


Figure 1.2: Vue générale prévue de la plateforme Virtual FabLab

1.9 Conclusion

Ce chapitre a présenté l'organisme d'accueil, le contexte général du projet, la problématique ainsi que les objectifs de la plateforme Virtual FabLab. Le projet se positionne à l'intersection du développement web, de la simulation 3D, de l'IoT et de la Data Science.

Le chapitre suivant sera consacré à l'analyse des besoins et à la conception du système. Il présentera les acteurs, les fonctionnalités attendues, l'architecture générale, les modules de la plateforme ainsi que les technologies retenues pour la réalisation.

CHAPITRE 2

ANALYSE DES BESOINS ET CONCEPTION

2.1 Introduction

Après avoir présenté le cadre général du projet, ce chapitre détaille l’analyse des besoins et la conception de la plateforme Virtual FabLab. L’objectif est de formaliser les fonctionnalités attendues, les contraintes de qualité, les acteurs du système, puis de proposer une architecture technique cohérente avec l’implémentation réalisée.

La conception retenue s’appuie sur une application web composée d’un frontend interactif, d’un backend applicatif, d’une base de données, d’un flux MQTT simulé et d’un module d’analyse de données.

2.2 Étude des besoins

L’étude des besoins vise à identifier les services que la plateforme doit fournir aux utilisateurs. Le système doit répondre à deux catégories principales d’utilisateurs : les étudiants et les administrateurs.

L’étudiant doit pouvoir accéder au FabLab virtuel, consulter les machines disponibles, réserver un créneau et tester un fichier de fabrication dans une simulation 3D. L’administrateur doit disposer d’un espace de pilotage lui permettant de gérer les machines, les utilisateurs, les réservations et les indicateurs de supervision.

Le système doit également être capable de recevoir des données simulées issues des machines, de les stocker et de les exploiter pour la détection d’anomalies et l’aide à la maintenance. Les données utilisées dans cette version sont simulées afin de valider l’architecture, les flux MQTT, le monitoring et le module d’analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

2.3 Besoins fonctionnels

2.3.1 Besoins de l’étudiant

Les fonctionnalités attendues pour l’étudiant sont les suivantes :

- créer un compte utilisateur ;
- s’authentifier et accéder à un espace personnel ;
- consulter les machines disponibles dans le FabLab ;
- visualiser le laboratoire virtuel en 3D ;
- sélectionner une machine et consulter ses informations ;

- effectuer une demande de réservation sur un créneau disponible ;
- consulter l'historique de ses réservations ;
- importer un fichier G-code ou NC compatible ;
- lancer, mettre en pause et réinitialiser une simulation ;
- modifier les informations de son profil.

2.3.2 Besoins de l'administrateur

Les fonctionnalités attendues pour l'administrateur sont les suivantes :

- consulter un tableau de bord global ;
- gérer les utilisateurs et leurs rôles ;
- activer ou désactiver un compte utilisateur ;
- créer, modifier ou supprimer des machines ;
- modifier l'état et le placement 3D des machines ;
- consulter les demandes de réservation ;
- approuver, refuser ou supprimer une réservation ;
- superviser les données de télémétrie reçues ;
- suivre l'état MQTT du système ;
- consulter les alertes d'anomalies ;
- consulter les scores de santé et les recommandations de maintenance.

2.4 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues de la solution :

- **Sécurité** : les fonctionnalités sensibles doivent être protégées par authentification et contrôle de rôle.
- **Ergonomie** : l'interface doit être claire, lisible et adaptée à un usage pédagogique.
- **Maintenabilité** : le code doit être organisé en modules séparant les responsabilités.
- **Évolutivité** : la solution doit permettre l'ajout de nouveaux types de machines ou de nouveaux indicateurs.
- **Performance** : la scène 3D et la simulation doivent rester fluides dans un navigateur moderne.

- **Traçabilité** : les données de télémétrie doivent être conservées afin d'alimenter l'historique et les analyses.
- **Interopérabilité** : les échanges entre frontend et backend doivent être réalisés à travers des API structurées.

2.5 Acteurs du système

Le système met en interaction plusieurs acteurs :

- **Étudiant** : utilisateur inscrit qui consulte les machines, réserve des créneaux et lance des simulations.
- **Administrateur** : utilisateur disposant de droits avancés pour gérer la plateforme et superviser les machines.
- **Broker MQTT** : composant intermédiaire chargé de transmettre les messages de télémétrie simulée.
- **Machine simulée** : source logique des données de fonctionnement publiées dans le système.

2.6 Cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les interactions principales entre les acteurs et la plateforme.

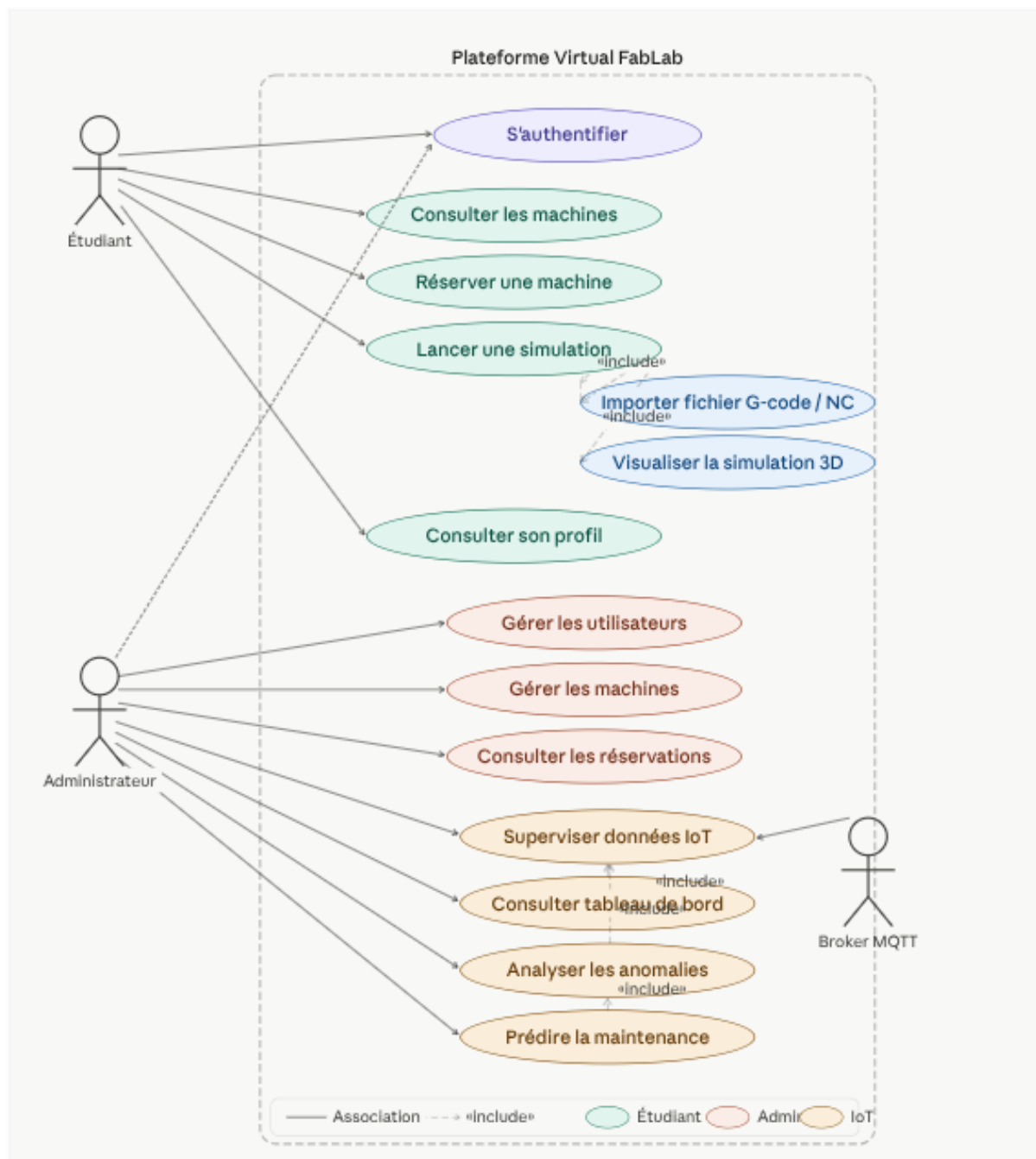


Figure 2.1: Diagramme de cas d'utilisation de la plateforme Virtual FabLab

Les cas d'utilisation principaux sont l'authentification, la consultation des machines, la réservation, la simulation, la gestion administrative, la supervision IoT et l'analyse des anomalies.

2.7 Architecture générale

L'architecture générale de la plateforme repose sur une séparation claire entre l'interface utilisateur, les services backend, la base de données, le flux MQTT et le module Data Science.

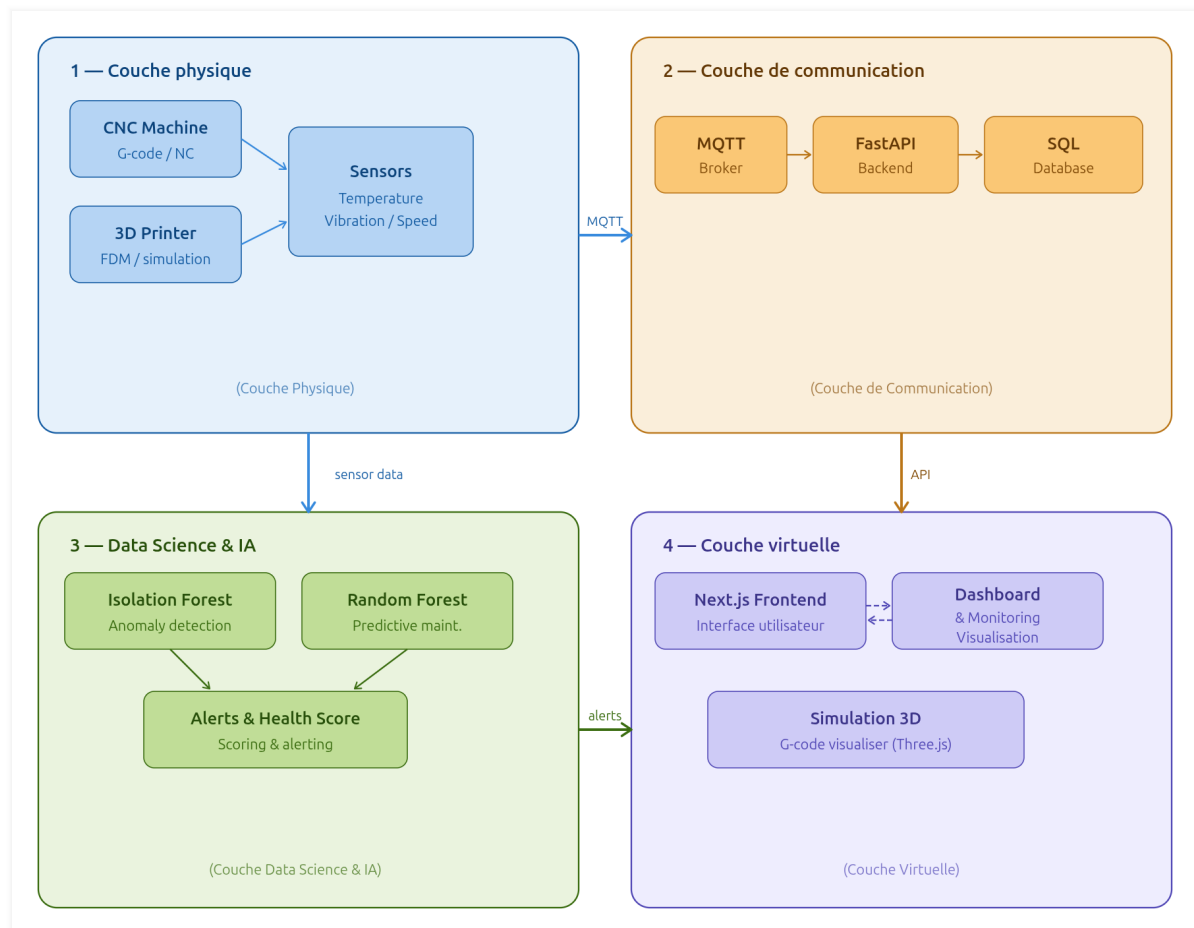


Figure 2.2: Architecture générale de la plateforme Virtual FabLab

Le frontend communique avec le backend à travers des requêtes HTTP. Le backend expose les API, gère la base de données SQLite et lance un abonné MQTT au démarrage de l'application. Les messages reçus sont transformés en mesures de télémétrie puis exploités par les services de monitoring et de prédiction.

2.8 Description des modules

La plateforme est structurée autour des modules suivants :

- **Module d'authentification** : inscription, connexion, profil, changement de mot de passe et génération de jetons JWT.
- **Module utilisateurs** : liste, recherche, changement de rôle et activation ou désactivation des comptes.
- **Module machines** : gestion des types de machines, des instances, de leurs états et de leurs coordonnées dans le laboratoire 3D.
- **Module réservations** : création des demandes, calcul des créneaux disponibles, an-

nulation et validation par l'administrateur.

- **Module FabLab 3D** : visualisation interactive de la salle et des machines avec sélection et consultation de leur état.
- **Module simulation** : importation et interprétation des fichiers G-code ou NC pour l'imprimante 3D et la CNC.
- **Module MQTT** : réception des données simulées publiées sur les topics machines.
- **Module Data Science** : détection d'anomalies avec Isolation Forest, estimation du risque de panne avec Random Forest Classifier, calcul du score de santé, niveau de risque et recommandations par logique de règles.
- **Module notifications** : notification des étudiants après validation ou refus d'une réservation.

2.9 Diagramme de classes

Le diagramme de classes représente les entités principales utilisées dans la base de données et leurs relations : utilisateur, machine, type de machine, réservation, mesure capteur et notification.

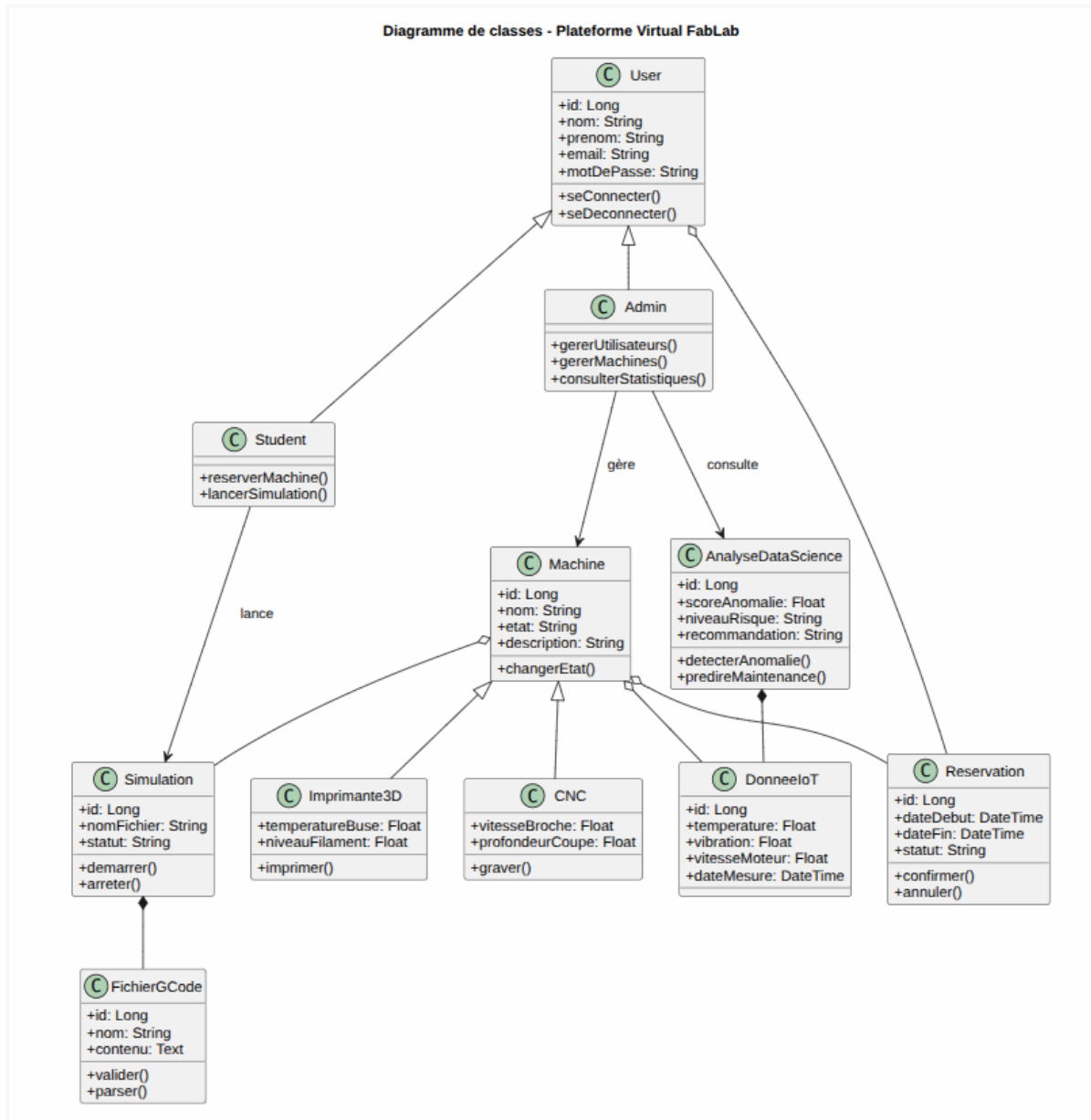


Figure 2.3: Diagramme de classes de la plateforme Virtual FabLab

La conception des données distingue les types de machines des instances de machines. Cette séparation permet d'associer un modèle 3D, un schéma de capteurs et des informations techniques à un type, tout en laissant chaque instance posséder son propre nom, état et emplacement.

2.10 Conception du système

2.10.1 Conception de l'authentification

Le système d'authentification repose sur des jetons JWT. Après connexion, le backend renvoie un jeton d'accès utilisé par le frontend pour appeler les routes protégées. Le rôle de

l'utilisateur permet de distinguer les droits d'un étudiant et ceux d'un administrateur.

2.10.2 Conception de la gestion des machines

Chaque machine possède un type, un nom unique, un état, des notes et des paramètres de placement dans la scène 3D. Les machines peuvent être consultées par les utilisateurs, tandis que leur création, modification ou suppression est réservée à l'administrateur.

2.10.3 Conception de la réservation

Le système de réservation repose sur des créneaux d'une heure entre 8h00 et 20h00. Une demande est créée avec l'état **pending**. L'administrateur peut ensuite l'approuver ou la refuser. Les conflits sont évités en vérifiant les réservations actives pour la même machine et le même intervalle.

2.10.4 Conception de la télémétrie

Dans l'implémentation actuelle, la télémétrie simulée est reçue sous forme de messages JSON publiés via MQTT. Le backend associe chaque message à une machine existante, enregistre les mesures et expose l'historique à travers les routes de monitoring.

2.10.5 Conception de l'analyse intelligente

Le module Data Science combine des modèles entraînés avec scikit-learn et une logique par règles. La détection d'anomalies s'appuie sur un modèle *Isolation Forest*, tandis que l'estimation du risque de panne ou de maintenance repose sur un *Random Forest Classifier*. Une logique complémentaire par règles est utilisée pour consolider le score de santé, le niveau de risque et les recommandations de maintenance, notamment lorsque les modèles ne sont pas disponibles ou lorsqu'une lecture doit être interprétée de manière explicable.

Les variables analysées sont la température, la vibration, la durée d'utilisation exprimée sous forme `usage_duration` puis convertie en `usage_hours`, la vitesse du moteur `motor_speed` et l'historique des erreurs signalées. Les sorties produites par le module sont le statut d'anomalie, la probabilité indicative de panne, le score de santé, le niveau de risque et une recommandation de maintenance.

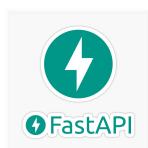
2.11 Technologies utilisées

2.11.1 Next.js



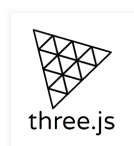
Next.js est un framework basé sur React permettant de développer des applications web modernes, modulaires et performantes. Dans ce projet, il constitue la base du frontend. Il organise les pages de l'application, les composants d'interface, les appels API et les vues interactives destinées aux étudiants et aux administrateurs.

2.11.2 FastAPI



FastAPI est un framework Python destiné à la création d'API web rapides et structurées. Il est utilisé pour implémenter le backend de la plateforme : authentification, gestion des machines, réservations, monitoring, réception MQTT et services d'analyse.

2.11.3 Three.js



Three.js est une bibliothèque JavaScript permettant de créer des scènes 3D dans le navigateur. Elle est utilisée pour afficher le laboratoire virtuel, charger les modèles GLB des machines et animer les trajectoires issues des fichiers de fabrication.

2.11.4 Blender



Blender est un logiciel de modélisation 3D open source. Il intervient dans la préparation et l'optimisation des modèles 3D utilisés dans la plateforme, notamment ceux de l'imprimante 3D et de la machine CNC.

2.11.5 MQTT



MQTT est un protocole léger de communication publish/subscribe très utilisé dans les systèmes IoT. Dans ce projet, il permet de simuler l'envoi de mesures machines vers le backend à l'aide d'un simulateur MQTT. Les messages sont publiés sur des topics propres à chaque machine, selon le motif suivant :

```
fablab/machines/{machine_id}/data
```

2.11.6 Python



Python est utilisé côté backend pour développer les services applicatifs et les traitements de données. Il facilite également l'intégration des bibliothèques de Data Science et l'entraînement des modèles.

2.11.7 Tailwind CSS



Tailwind CSS est un framework CSS utilitaire permettant de construire rapidement des interfaces cohérentes et responsives. Dans la plateforme, il est utilisé pour structurer les pages, les cartes, les tableaux, les formulaires et les tableaux de bord.

2.11.8 Scikit-learn



Scikit-learn est une bibliothèque Python dédiée au Machine Learning. Elle est utilisée dans ce projet pour entraîner et exploiter un modèle de détection d'anomalies de type Isolation Forest et un Random Forest Classifier pour l'estimation du risque de maintenance.

2.11.9 Visual Studio Code



Visual Studio Code est l'environnement de développement utilisé pour organiser et modifier les différents modules du projet : frontend, backend, simulation, scripts Python et fichiers LaTeX du rapport.

2.11.10 Technologies et équipements IoT

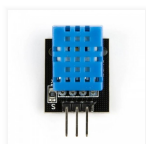
Cette partie présente les composants matériels et les équipements de fabrication numérique liés à la dimension IoT du FabLab. Dans la version actuelle, ils servent de référence pour concevoir les mesures et les scénarios de supervision. La collecte effective n'est pas encore réalisée par des capteurs physiques : elle est simulée par MQTT afin de valider les flux, le stockage et l'analyse avant une intégration matérielle future.

ESP8266



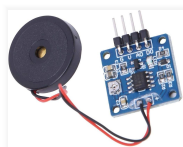
L'ESP8266 est un microcontrôleur doté d'une connectivité Wi-Fi intégrée, largement utilisé dans les applications de l'Internet des Objets. Dans un contexte FabLab, il peut servir d'interface entre les capteurs physiques et la plateforme de supervision. Il permet de collecter des mesures, de les traiter localement puis de les transmettre vers un serveur ou un broker MQTT. Son faible coût et sa facilité de programmation en font un choix adapté aux prototypes académiques et industriels légers.

Capteur de température



Le capteur de température permet de mesurer l'échauffement d'un environnement, d'un composant électronique ou d'une machine en fonctionnement. Dans le cadre d'un FabLab intelligent, cette donnée est utile pour surveiller les conditions de travail et prévenir les situations anormales. Une température excessive peut indiquer une surcharge, un défaut de ventilation ou une dégradation progressive d'un équipement. Lors d'une connexion future à des capteurs réels, ces mesures pourront alimenter des tableaux de bord et des modèles de maintenance prédictive.

Capteur de vibration



Le capteur de vibration mesure les oscillations mécaniques produites par une machine pendant son fonctionnement. Ces données sont essentielles pour les équipements de fabrication numérique, car des vibrations inhabituelles peuvent révéler un désalignement, une usure ou un défaut mécanique. Dans une approche de maintenance préventive, l'analyse de ces signaux aide à identifier les anomalies avant l'apparition d'une panne. Les données de vibration constituent ainsi un indicateur pertinent de l'état de santé des machines.

Imprimante 3D



L'imprimante 3D est un équipement de fabrication additive permettant de produire des objets physiques à partir de modèles numériques. Elle fonctionne généralement par dépôt progressif de matière selon des trajectoires définies dans un fichier G-code. Dans un FabLab, elle constitue un outil pédagogique et technique essentiel pour le prototypage rapide. Son intégration à une plateforme IoT permet de suivre son état, sa température, sa progression et ses éventuelles anomalies de fonctionnement.

Machine CNC



La machine CNC est un équipement de fabrication soustractive commandé numériquement. Elle exécute des opérations d'usinage à partir d'instructions précises, souvent décrites dans des fichiers NC ou G-code. Dans un FabLab, elle permet de travailler différents matériaux avec une grande précision. La supervision IoT d'une CNC peut porter sur son état, ses vibrations, sa durée d'utilisation et ses paramètres de fonctionnement afin d'améliorer la sécurité, la traçabilité et la maintenance.

2.12 Conclusion

Ce chapitre a présenté l'analyse des besoins, les acteurs, les cas d'utilisation et la conception générale de la plateforme. Il a également décrit les technologies retenues et leur rôle dans le projet.

Le chapitre suivant sera consacré à la réalisation et à l'implémentation. Il détaillera l'organisation concrète du frontend et du backend, les modules développés, la simulation 3D, le flux MQTT et les interfaces de supervision.

CHAPITRE 3

RÉALISATION ET IMPLÉMENTATION

3.1 Introduction

Ce chapitre présente la réalisation technique de la plateforme Virtual FabLab. Contrairement au chapitre précédent, qui décrit l'analyse et la conception, cette partie se concentre sur l'implémentation effective du projet : organisation du code, architecture frontend/backend, modules fonctionnels, simulation 3D, communication MQTT et tableau de bord de supervision.

La plateforme réalisée repose sur un backend FastAPI, une interface Next.js et des modules spécialisés pour la visualisation 3D, l'interprétation des fichiers de fabrication et l'analyse des données de télémétrie. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

3.2 Architecture frontend/backend

L'implémentation adopte une architecture séparant clairement le frontend et le backend. Le frontend prend en charge l'interface utilisateur, la navigation, les scènes 3D et les tableaux de bord. Le backend fournit les API, la sécurité, la base de données, l'ingestion MQTT et les traitements analytiques.

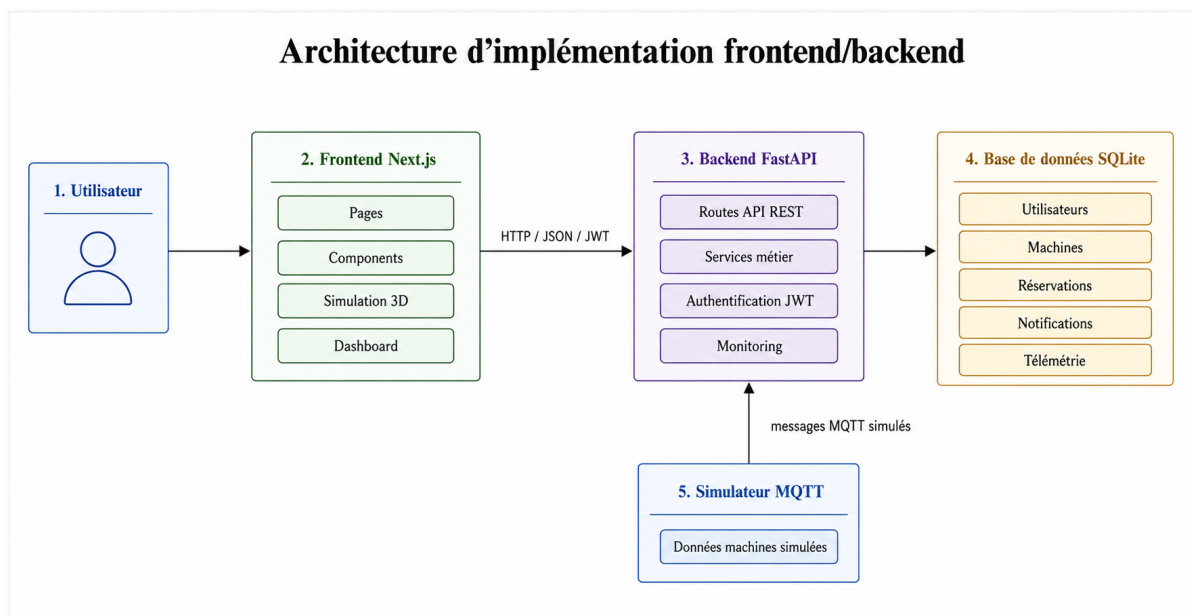


Figure 3.1: Architecture d'implémentation frontend/backend

Le backend est initialisé dans le point d'entrée FastAPI. Au démarrage, il initialise la base de données et lance le service abonné MQTT. Les routes sont ensuite organisées par

domaine fonctionnel : authentification, machines, réservations, administration, monitoring, simulation, notifications et intelligence artificielle.

3.3 Organisation des dossiers

Le projet est organisé en trois parties principales : le frontend, le backend et le rapport LaTeX. Cette organisation facilite la maintenance et permet d’isoler les responsabilités.

Table 3.1: *Organisation générale du projet*

Dossier	Rôle
Frontend/app	Pages principales de l’application Next.js : accueil, lab 3D, simulation, dashboard, réservations, utilisateurs et profil.
Frontend/components	Composants réutilisables : layout, authentification, scène 3D, simulation, tableaux de bord et composants UI.
Frontend/lib	Fonctions utilitaires, appels API, traductions et moteur d’interprétation des fichiers G-code/NC.
backend/app/routes	Définition des endpoints FastAPI par domaine fonctionnel.
backend/app/services	Logique métier : authentification, machines, réservations, télémétrie, MQTT, anomalies et prédiction.
backend/app/models	Modèles SQLAlchemy représentant les entités persistées.
backend/scripts	Scripts de simulation MQTT et d’entraînement des modèles de Machine Learning.
Rapport_PFE	Sources LaTeX du rapport académique.

3.4 Implémentation du backend

3.4.1 API FastAPI

Le backend expose une API REST structurée autour de plusieurs routeurs. Chaque routeur regroupe les endpoints liés à un domaine précis. Cette organisation améliore la lisibilité du code et facilite l’évolution de la plateforme.

Les principales familles d’endpoints sont :

- `/auth` pour l’inscription, la connexion et le profil ;
- `/machines` pour la gestion administrative des machines ;
- `/lab/machines` pour l’affichage public des machines du FabLab ;
- `/reservations` pour les demandes de réservation ;

- /admin pour la gestion des utilisateurs et des réservations ;
- /monitoring pour la télémétrie et l'état MQTT ;
- /simulation pour l'état de simulation ;
- /admin/ai pour les analyses intelligentes réservées à l'administrateur.

3.4.2 Base de données

La persistance est assurée par SQLite avec SQLAlchemy. Les modèles principaux sont les utilisateurs, les types de machines, les machines, les réservations, les mesures capteurs et les notifications.

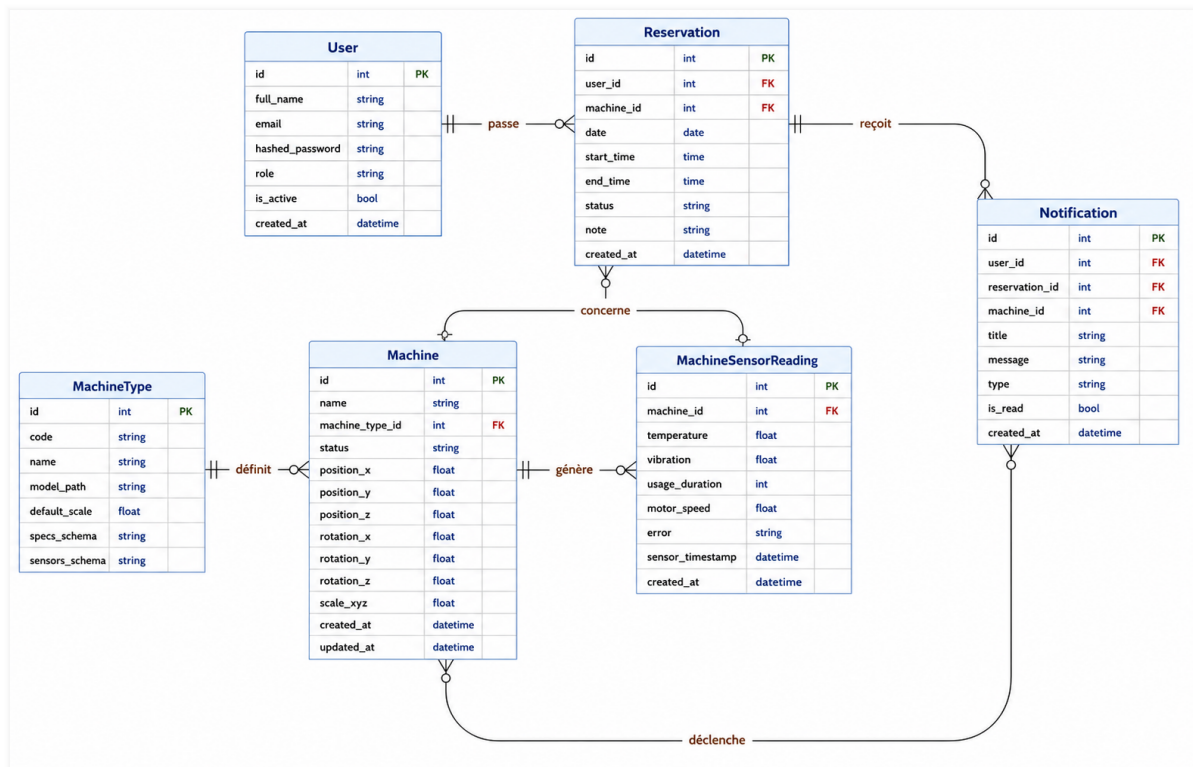


Figure 3.2: Schéma logique de la base de données

3.5 Système d'authentification

L'authentification est basée sur des jetons JWT. Lorsqu'un utilisateur se connecte, le backend vérifie son email et son mot de passe hashé, puis génère un jeton d'accès. Le frontend conserve ce jeton et l'ajoute dans l'en-tête **Authorization** lors des appels aux routes protégées.

Deux rôles sont définis dans le système :

- **student** : rôle par défaut pour les utilisateurs inscrits ;
- **admin** : rôle permettant l'accès aux interfaces et endpoints d'administration.

3.6 Gestion des utilisateurs et de l'administration

L'espace administrateur permet de consulter les utilisateurs, de rechercher un compte, de filtrer par rôle ou par état, puis de modifier les droits. L'administrateur peut également activer ou désactiver un compte.

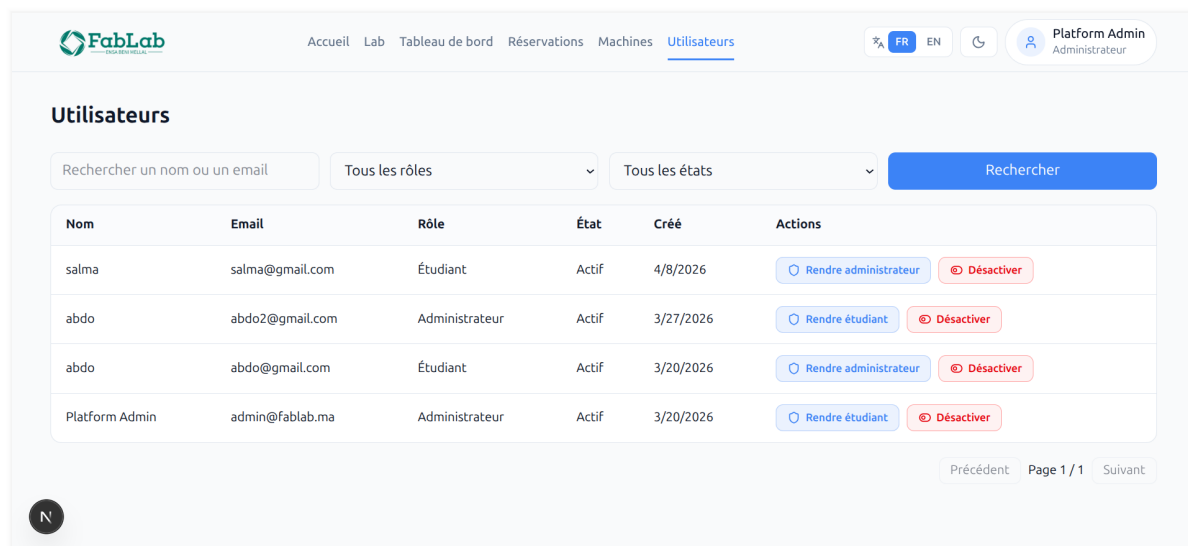


Figure 3.3: Interface de gestion des utilisateurs

L'accès à ces fonctionnalités est protégé par une dépendance backend qui vérifie que l'utilisateur courant possède le rôle administrateur.

3.7 Gestion des machines

Le module machines permet de gérer les instances du FabLab. Chaque machine possède un type, un nom, un état, des notes et des paramètres de placement dans la scène 3D : position, rotation et échelle.

Les types de machines initialisés par défaut incluent notamment :

- une imprimante 3D associée au modèle `/models/3d_printer.glb` ;
- une machine CNC associée au modèle `/models/CNC.glb` ;

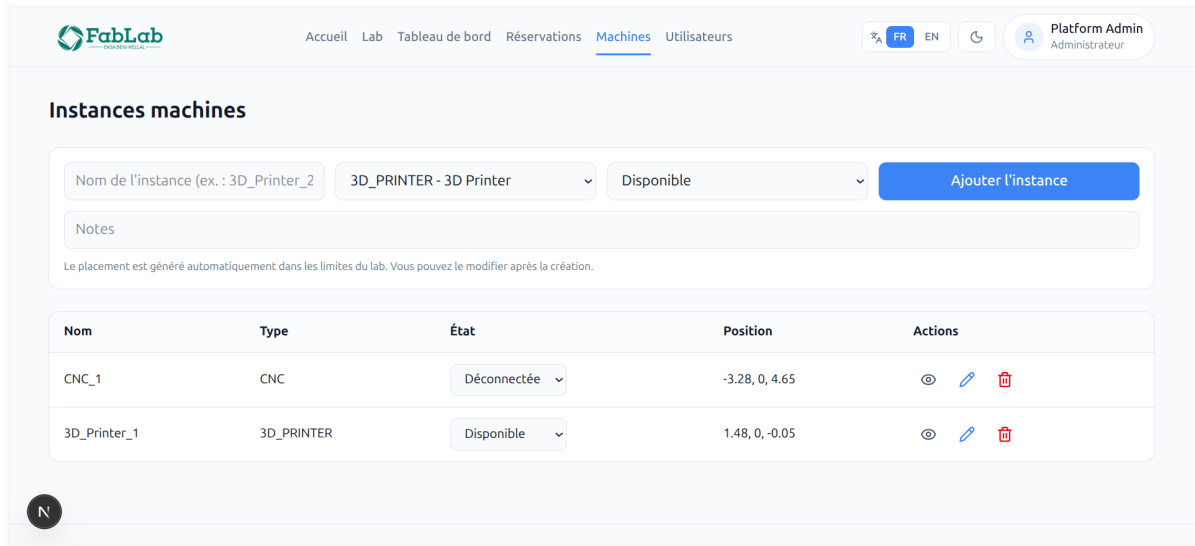


Figure 3.4: Interface de gestion des machines

3.8 Système de réservation

Le système de réservation permet à l'étudiant de choisir une machine, une date et un créneau horaire. Les créneaux sont générés entre 8h00 et 20h00 avec une durée d'une heure. Le backend vérifie les conflits afin d'empêcher deux réservations actives sur la même machine et le même intervalle.

Une demande de réservation est créée avec l'état **pending**. L'administrateur peut ensuite approuver ou refuser cette demande. Lorsqu'une décision est prise, une notification est créée pour l'utilisateur concerné.

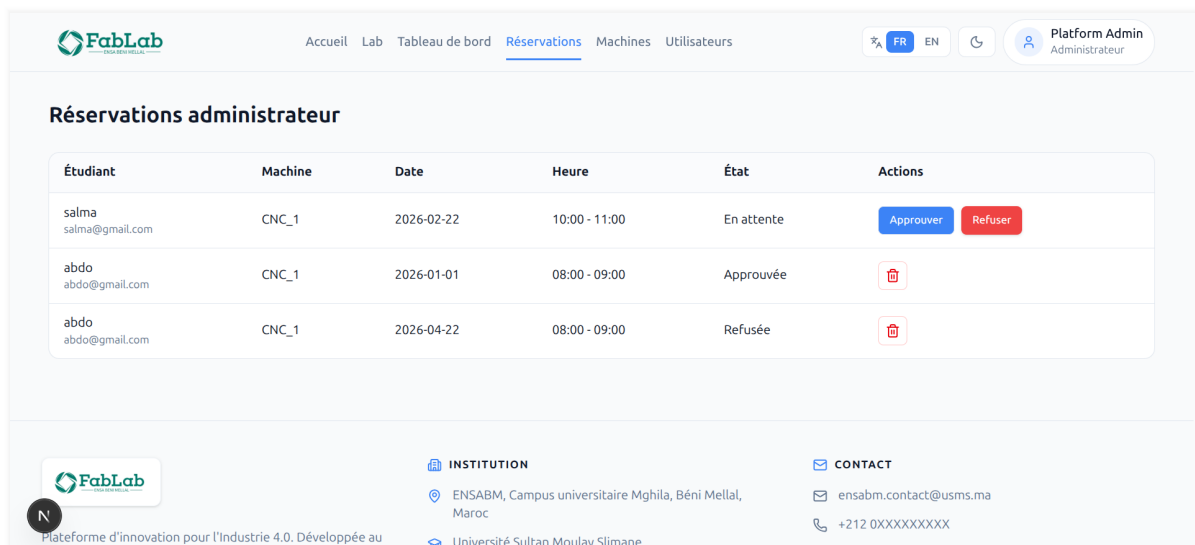


Figure 3.5: Interface de réservation des machines

3.9 FabLab virtuel en 3D

La page du laboratoire virtuel affiche une scène 3D représentant l'espace du FabLab. Les machines sont chargées dynamiquement depuis le backend, ce qui permet d'afficher les machines réellement enregistrées dans la base de données.

Chaque machine possède un modèle GLB, une position, une rotation et une échelle. L'utilisateur peut sélectionner une machine dans la scène afin de consulter son état, ses mesures récentes et ses actions disponibles : consulter les détails, ouvrir la simulation ou réserver.

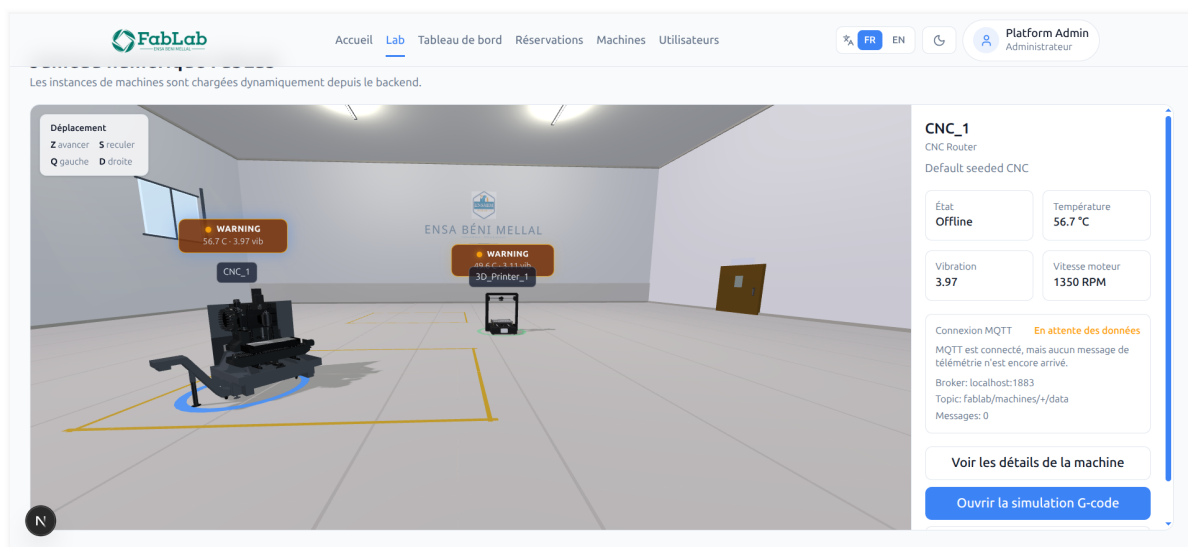


Figure 3.6: Vue 3D du FabLab virtuel

3.10 Simulation CNC

La simulation CNC repose sur l'interprétation de fichiers NC ou G-code adaptés à une machine de fraisage ou de découpe. Le module détecte les commandes de mouvement, les vitesses d'avance, les arcs, les commandes de broche et certains signaux caractéristiques de la CNC.

Les trajectoires sont transformées en mouvements simulables dans la scène 3D. Les déplacements rapides et les opérations de coupe sont représentés différemment afin de distinguer les phases de positionnement et les phases d'usinage.

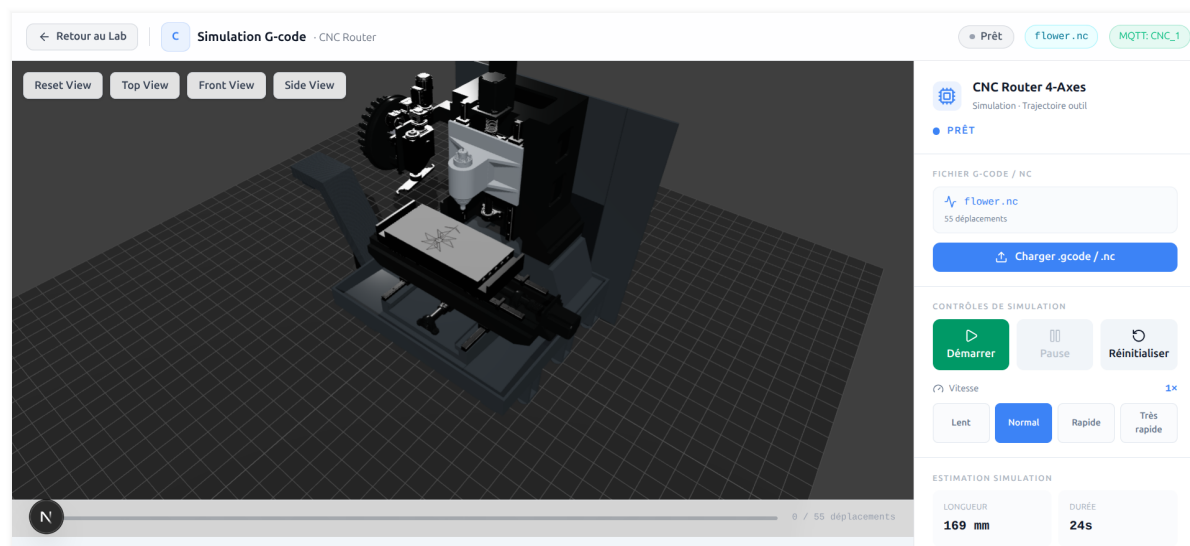


Figure 3.7: Simulation d'une trajectoire CNC

3.11 Simulation de l'imprimante 3D

La simulation de l'imprimante 3D prend en charge les fichiers G-code contenant les déplacements, l'axe d'extrusion, les vitesses d'avance et certaines commandes propres à l'impression 3D. Le système distingue les déplacements sans extrusion des segments imprimés.

La scène affiche progressivement la trajectoire d'impression et peut représenter les couches de l'objet. Le panneau latéral permet de suivre la progression, la position courante, la vitesse de lecture, la température simulée et les informations associées à la télémétrie lorsque la machine sélectionnée est liée à une instance enregistrée dans le backend.

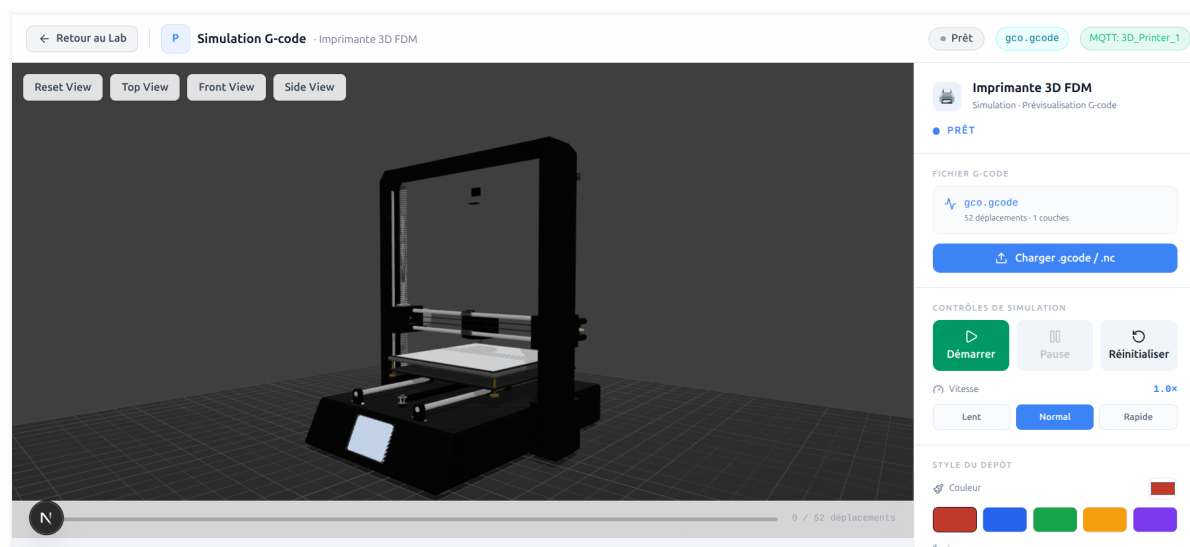


Figure 3.8: Simulation d'un fichier G-code pour imprimante 3D

3.12 Interprétation des fichiers G-code et NC

L'interprétation des fichiers est réalisée côté frontend afin de fournir un retour immédiat à l'utilisateur. Les lignes sont normalisées, les commentaires sont ignorés et les tokens de type G, M, X, Y, Z, F, E, I, J ou R sont analysés.

Le système vérifie également la compatibilité du fichier avec la machine sélectionnée. Par exemple, un fichier contenant des commandes de broche ou de refroidissement est considéré comme non compatible avec l'imprimante 3D, tandis qu'un fichier contenant des commandes d'extrusion est rejeté pour la CNC.

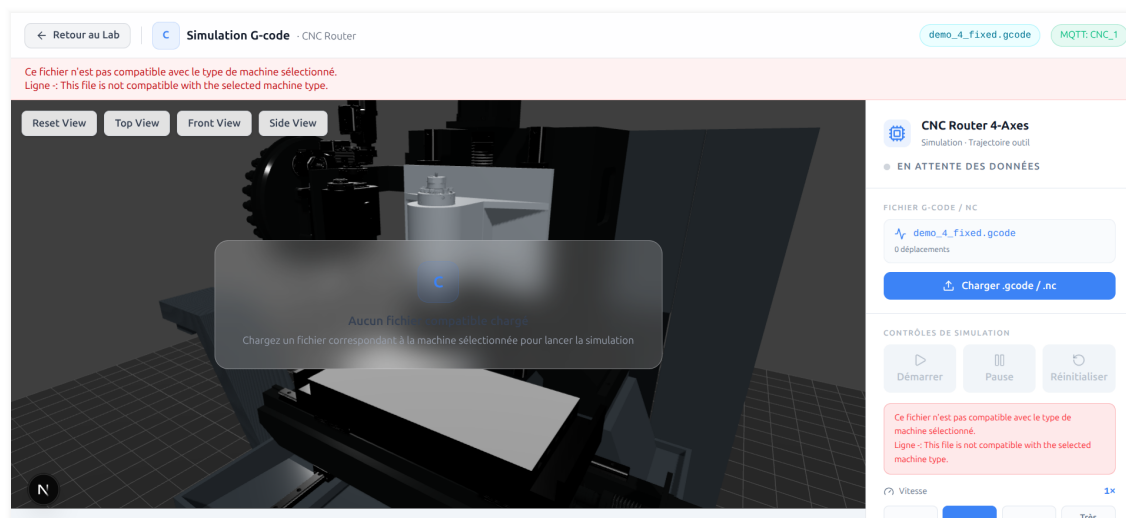


Figure 3.9: Processus d'interprétation et de validation des fichiers G-code/NC

3.13 Télémétrie simulée avec MQTT

La télémétrie est simulée par un script Python qui publie périodiquement des messages MQTT. Ce choix permet de valider le flux complet sans dépendre, à ce stade, d'une installation physique de capteurs. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels. Chaque message est envoyé sur un topic de la forme :

`fablab/machines/{machine_id}/data`

Le message contient l'identifiant de la machine, la température, la vibration, la durée d'utilisation, la vitesse du moteur, une éventuelle erreur et un horodatage. Le backend s'abonne au motif `fablab/machines/+/data`, analyse les messages JSON et enregistre les mesures dans la base de données SQLite.

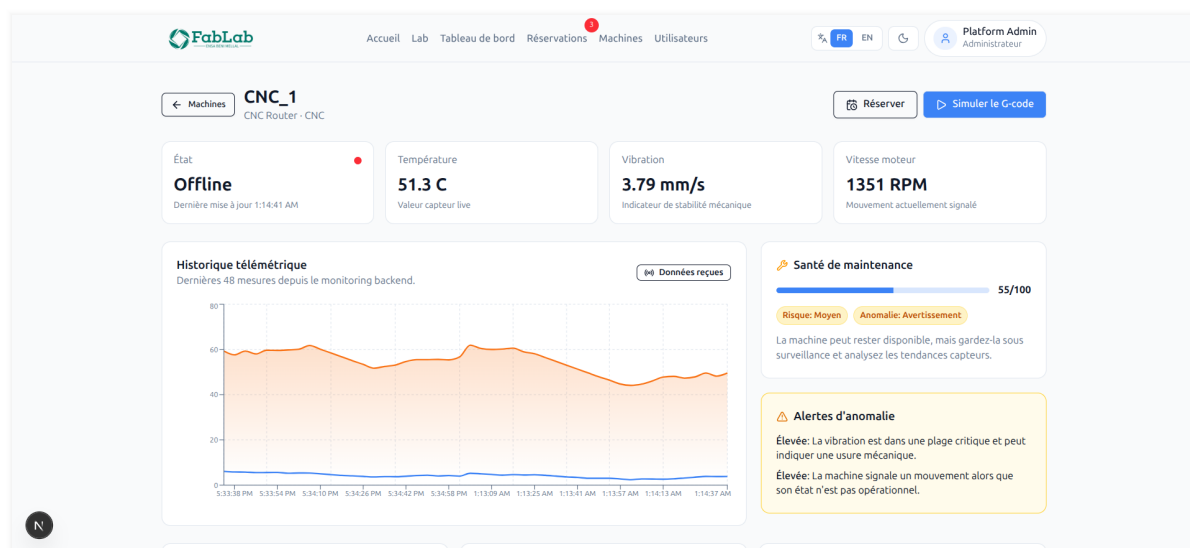


Figure 3.10: Flux MQTT implémenté dans la plateforme

3.14 Tableau de bord Data Science

Le tableau de bord administrateur centralise les informations de supervision. Il affiche l'état de la connexion MQTT, la fraîcheur des données, le nombre d'utilisateurs, le nombre de machines, les réservations en attente et les résultats de monitoring intelligent.

Le module Data Science exploite les mesures simulées reçues via MQTT afin de produire :

- un statut d'anomalie par machine ;
- un score de santé sur 100 ;
- un niveau de risque de maintenance ;
- une probabilité indicative de panne ;
- une recommandation de maintenance ;
- des facteurs explicatifs tels que température, vibration, durée d'utilisation, vitesse du moteur et erreurs récentes.

Sur le plan technique, la détection d'anomalies utilise un modèle *Isolation Forest*. L'estimation du risque de panne ou de maintenance s'appuie sur un *Random Forest Classifier*. Le score de santé, le niveau de risque et les recommandations sont complétés par une logique par règles, afin de rendre les résultats exploitables même lorsque les modèles entraînés ne sont pas disponibles ou lorsque des seuils métier doivent être pris en compte.

Les caractéristiques analysées sont `temperature`, `vibration`, `usage_duration` convertie en `usage_hours`, `motor_speed` et l'historique des erreurs. Les sorties exposées au tableau

de bord et aux endpoints d'analyse sont le statut d'anomalie, la probabilité indicative de panne, le score de santé, le niveau de risque et une recommandation de maintenance.

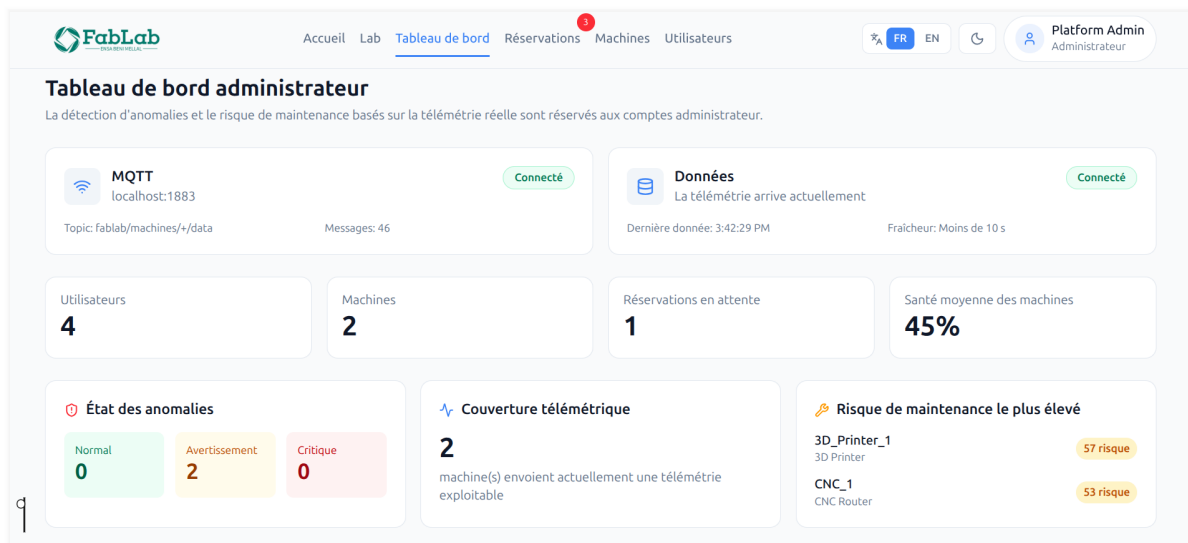


Figure 3.11: *Tableau de bord administrateur et monitoring intelligent*

3.15 Alertes d'anomalies et score de santé

La détection des anomalies repose sur deux niveaux. Le premier exploite le modèle *Isolation Forest* lorsque les fichiers de modèles entraînés sont disponibles. Le second utilise des règles et des seuils adaptés au type de machine, par exemple pour la température, la vibration, la durée d'utilisation, la vitesse du moteur et les erreurs signalées. Cette combinaison permet de produire un résultat exploitable dans le prototype tout en conservant une interprétation simple des principaux facteurs de risque.

Le risque de maintenance est estimé par le *Random Forest Classifier* lorsqu'il est disponible, puis consolidé avec la logique de règles. Le score de santé est calculé à partir du risque estimé. Un score élevé indique une machine considérée comme saine dans le cadre des données simulées, tandis qu'un score faible signale un besoin de surveillance ou d'intervention. Les cartes de prédiction du tableau de bord permettent de consulter rapidement les machines présentant le risque le plus important.



Figure 3.12: Cartes de prédiction et score de santé des machines

3.16 Graphiques et historique de télémétrie

Les pages de détail des machines exploitent l'historique des mesures stockées pour afficher l'évolution de certains indicateurs, notamment la température. Cette visualisation permet à l'administrateur de suivre l'état récent d'une machine et de relier les alertes à une série temporelle.



Figure 3.13: Historique des mesures d'une machine

3.17 Communication entre frontend et backend

La communication entre le frontend et le backend est centralisée dans une fonction utilitaire d'appel API. Cette fonction construit l'URL du backend, ajoute l'en-tête **Content-Type**, insère le jeton d'authentification lorsque nécessaire et traite les erreurs HTTP.

Cette couche simplifie l'usage des API dans les pages Next.js. Elle permet également de maintenir une cohérence dans la gestion des réponses, des erreurs et des routes protégées.

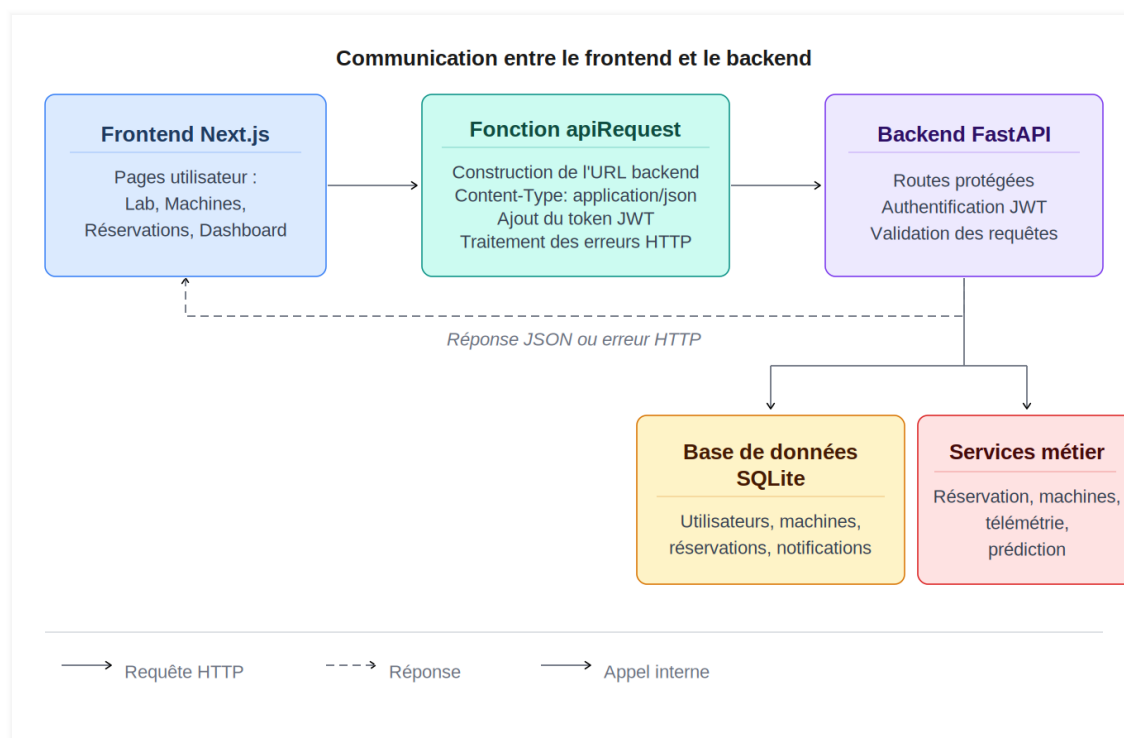


Figure 3.14: *Communication entre le frontend et le backend*

3.18 Interfaces réalisées

Les interfaces réalisées couvrent les principaux parcours utilisateurs : accueil, connexion, inscription, laboratoire 3D, simulation, réservation, profil, gestion des machines, gestion des utilisateurs, réservations administrateur et tableau de bord. Les captures présentées dans les sections précédentes illustrent les écrans les plus représentatifs de ces parcours.

3.19 Conclusion

Ce chapitre a présenté la réalisation technique de la plateforme Virtual FabLab. L'implémentation couvre la gestion des utilisateurs, des machines et des réservations, la visualisation du laboratoire en 3D, la simulation CNC et imprimante 3D, l'ingestion MQTT et le tableau de bord Data Science.

Le chapitre suivant approfondira l'exploitation des données IoT et l'intégration de la Data Science, en détaillant les données utilisées, les modèles de détection, les recommandations de maintenance et les limites de l'approche adoptée.

CHAPITRE 4

VALIDATION ET RÉSULTATS

4.1 Introduction

Ce chapitre présente la validation fonctionnelle de la plateforme **Virtual FabLab**, les résultats obtenus, ainsi que les limites actuelles du travail réalisé. Après la description de l'implémentation dans le chapitre précédent, il s'agit ici de vérifier que les principales fonctionnalités répondent aux besoins définis : consultation du laboratoire virtuel, gestion des machines, réservation, simulation 3D, authentification, administration et communication entre le frontend et le backend.

La validation effectuée reste adaptée au contexte d'un projet de fin d'études. Elle porte principalement sur le bon fonctionnement des parcours utilisateurs et administrateurs, ainsi que sur la cohérence des échanges entre les différentes couches de l'application. Les parties liées à l'IoT et à la Data Science sont abordées avec prudence, car elles reposent sur des données simulées par un simulateur MQTT. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

4.2 Validation fonctionnelle de la plateforme

4.2.1 Tests du frontend

Les tests du frontend ont porté sur les interfaces développées avec Next.js. L'objectif était de vérifier que les pages principales sont accessibles, lisibles et cohérentes avec les rôles des utilisateurs. Les interfaces testées incluent notamment la page d'accueil, le laboratoire 3D, la page des machines, les réservations, le tableau de bord administrateur et les pages de gestion.

La navigation a également été vérifiée afin de s'assurer que l'utilisateur peut passer d'un module à l'autre sans rupture de parcours. Les redirections vers la page de connexion sont déclenchées lorsque l'utilisateur n'est pas authentifié, tandis que les pages réservées à l'administrateur restent protégées pour les utilisateurs étudiants.

La partie simulation 3D a été validée à travers le chargement de la scène du FabLab, la sélection des machines et la visualisation des actions associées. Les modules de simulation CNC et imprimante 3D ont été testés avec des fichiers de démonstration afin de vérifier l'affichage des trajectoires, la progression de la simulation et la gestion des fichiers non valides. Ces tests confirment le caractère opérationnel de la simulation dans un objectif pédagogique, même si elle ne reproduit pas encore l'ensemble des phénomènes physiques

réels.

4.2.2 Tests du backend FastAPI

La validation du backend FastAPI a concerné l'authentification, les routes protégées, les API REST et la gestion des erreurs. Le mécanisme d'authentification repose sur un jeton JWT généré lors de la connexion. Ce jeton permet ensuite d'accéder aux routes protégées selon le rôle de l'utilisateur.

Les tests effectués ont permis de vérifier les comportements suivants :

- création d'un compte utilisateur avec des données valides ;
- connexion et récupération du jeton JWT ;
- accès au profil de l'utilisateur connecté ;
- protection des routes nécessitant une authentification ;
- restriction des routes administrateur aux utilisateurs autorisés ;
- création, consultation et traitement des réservations ;
- consultation et gestion des machines selon les permissions.

La validation des requêtes a également été prise en compte. Lorsque les données envoyées sont incorrectes ou incomplètes, le backend retourne des erreurs adaptées. Les principaux cas traités concernent les erreurs d'authentification, les accès interdits, les ressources inexistantes et les conflits liés aux réservations. Cette gestion permet au frontend d'afficher des messages compréhensibles et d'éviter les comportements incohérents.

4.2.3 Communication frontend/backend

La communication entre le frontend et le backend est centralisée dans la fonction utilitaire `apiRequest`. Cette fonction joue un rôle important dans l'organisation du code, car elle évite de répéter la logique d'appel API dans chaque page ou composant.

Son rôle consiste principalement à construire l'URL du backend, ajouter les en-têtes nécessaires aux échanges JSON, intégrer le token JWT lorsqu'il est disponible, puis traiter les réponses retournées par le backend. Lorsqu'une erreur HTTP est renvoyée, la fonction récupère le message d'erreur et le transforme en information exploitable par l'interface.

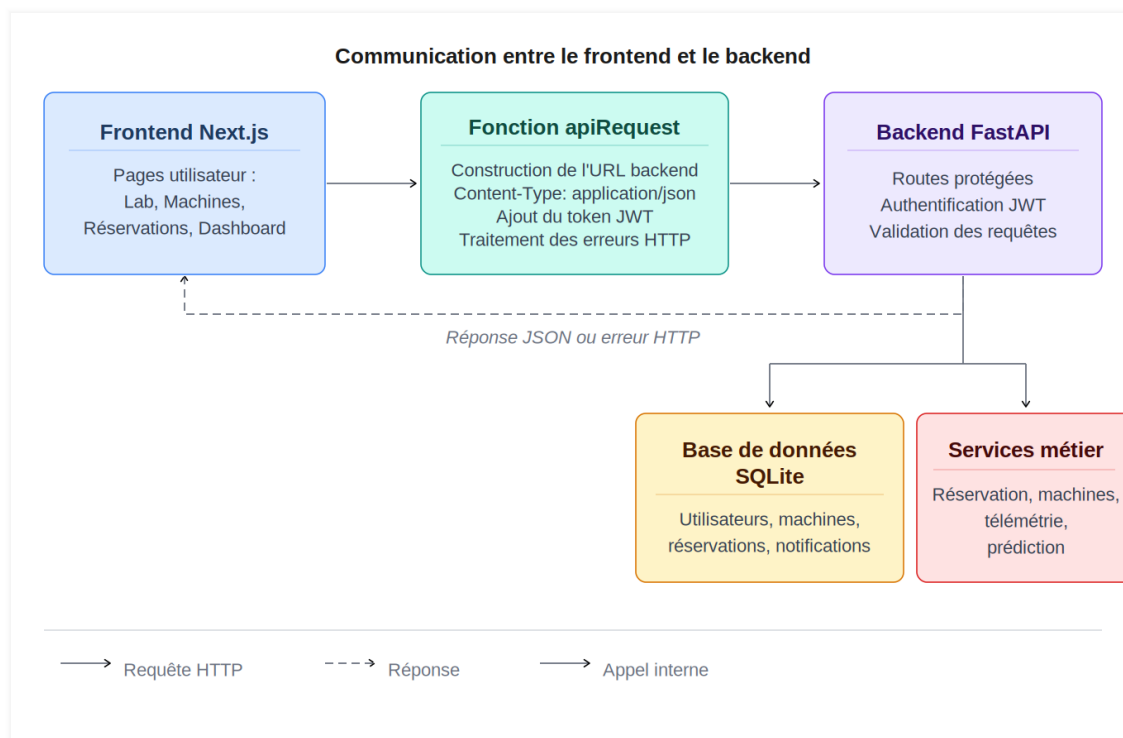


Figure 4.1: Schéma de communication entre l'interface Next.js et le backend FastAPI

La figure 4.1 illustre le principe général des échanges. Le frontend envoie une requête HTTP vers l'API FastAPI. Le backend valide la requête, applique la logique métier, interagit avec la base de données ou les services concernés, puis retourne une réponse JSON ou une erreur HTTP. Cette organisation rend la communication plus claire et facilite l'évolution de la plateforme.

4.2.4 Évaluation fonctionnelle du module Data Science

L'évaluation du module Data Science est une évaluation fonctionnelle du prototype. Elle ne vise pas à annoncer une précision industrielle, car les données utilisées sont simulées. L'objectif est de vérifier que le module est correctement intégré dans la chaîne applicative et qu'il produit des résultats cohérents avec les scénarios de télémétrie simulée.

Les vérifications réalisées portent sur les points suivants :

- les messages MQTT publiés par le simulateur sont reçus par le backend ;
- les mesures de télémétrie sont enregistrées dans la base de données SQLite ;
- l'endpoint de prédiction retourne un statut d'anomalie, une probabilité indicative de panne, un score de santé, un niveau de risque et une recommandation ;
- le tableau de bord affiche le score de santé, la probabilité de panne et les recommandations de maintenance ;

- des alertes d'anomalies sont générées lorsque des valeurs anormales apparaissent dans la télémétrie simulée.

Cette validation confirme l'intégration technique du module : ingestion MQTT, stockage, analyse et restitution dans l'interface administrateur. Elle reste toutefois fonctionnelle et ne doit pas être interprétée comme une mesure de performance prédictive sur machines réelles.

4.3 Résultats obtenus

Les résultats obtenus montrent que la plateforme développée constitue un prototype fonctionnel répondant aux objectifs principaux du projet. L'application web permet à l'utilisateur de consulter les machines, accéder au laboratoire virtuel, effectuer des réservations et visualiser des simulations liées aux machines numériques.

Du côté administrateur, la plateforme permet la gestion des machines, des utilisateurs et des réservations. La séparation entre le frontend Next.js et le backend FastAPI rend l'architecture plus claire et facilite la maintenance du projet. Les routes backend assurent la persistance des données, la sécurité et l'exposition des fonctionnalités sous forme d'API REST.

La simulation 3D est opérationnelle pour représenter le FabLab virtuel et illustrer le comportement de machines telles que la CNC et l'imprimante 3D. Elle apporte une dimension visuelle et pédagogique importante au projet, en permettant à l'utilisateur d'interagir avec les machines avant une utilisation réelle.

Enfin, le projet intègre un premier module intelligent fondé sur la télémétrie simulée. Ce module exploite Isolation Forest pour la détection d'anomalies, Random Forest Classifier pour l'estimation indicative du risque de panne, et une logique par règles pour le score de santé, le niveau de risque et les recommandations de maintenance. Il permet de démontrer l'orientation du système vers un FabLab connecté et intelligent sans prétendre à une validation industrielle complète.

4.4 Limites actuelles et perspectives

4.4.1 Limites des données et validation

Malgré les résultats obtenus, certaines limites doivent être mentionnées. La première concerne la télémétrie, qui reste simulée. Les données utilisées permettent de tester les flux, les interfaces, le stockage et le module d'analyse, mais elles ne remplacent pas des mesures

réelles issues de capteurs installés sur des machines physiques.

Ainsi, les résultats du module Data Science sont indicatifs. Ils servent à valider le prototype, la cohérence des traitements et l'intégration entre MQTT, la base de données, les endpoints de prédiction et le tableau de bord. Ils ne permettent pas, à ce stade, d'affirmer une précision industrielle ou une capacité de généralisation sur de vraies machines.

La deuxième limite concerne l'absence de connexion directe avec de vraies machines IoT. Les scénarios de communication MQTT et de supervision sont préparés, mais ils nécessitent encore une validation avec des équipements réels. De même, la simulation 3D ne reproduit pas totalement le comportement réel des machines, notamment les contraintes mécaniques, les vibrations, les variations thermiques ou les défauts de fabrication.

4.4.2 Perspectives d'évolution

Plusieurs perspectives peuvent être envisagées pour améliorer la plateforme :

- intégrer un broker MQTT réel et des messages provenant de machines physiques ;
- ajouter des capteurs IoT pour collecter des mesures de température, vibration ou vitesse ;
- enrichir la maintenance prédictive à partir de données historiques réelles ;
- améliorer le réalisme de la simulation CNC et imprimante 3D ;
- développer un dashboard analytique plus avancé avec des courbes, alertes et indicateurs de performance.

Ces perspectives montrent que la solution actuelle peut servir de base évolutive. Elle permet déjà de valider l'architecture, les flux MQTT simulés, le stockage de la télémétrie et les principaux parcours fonctionnels, tout en laissant une marge d'amélioration importante vers une plateforme plus connectée et plus intelligente.

4.5 Conclusion

Ce chapitre a présenté la validation fonctionnelle de la plateforme, les résultats obtenus ainsi que les limites actuelles du projet. Les tests réalisés confirment le fonctionnement des principaux modules : frontend, backend, authentification, gestion des machines, réservations, simulation 3D et communication API.

La plateforme Virtual FabLab répond ainsi aux objectifs essentiels du projet, tout en restant réaliste sur les limites liées à la simulation et à l'absence de machines IoT réelles.

Elle intègre déjà un premier module intelligent de supervision et de maintenance prédictive indicative, dont la validation industrielle nécessitera des données issues de capteurs réels.

CONCLUSION GÉNÉRALE

Ce projet de fin d'études avait pour objectif de concevoir et de réaliser une plateforme web de FabLab virtuel intelligent, destinée à faciliter la gestion des machines, la visualisation de l'environnement de fabrication et la supervision des équipements dans un contexte académique.

Les travaux réalisés ont permis de mettre en place une plateforme web fonctionnelle intégrant une simulation 3D du FabLab, un système de communication basé sur le protocole MQTT pour la réception des données de télémétrie, ainsi qu'un module Data Science dédié à l'analyse de l'état des machines. Un tableau de bord administrateur complète la solution en assurant le suivi des équipements, des réservations et des principaux indicateurs de supervision.

Dans la version actuelle, les données de télémétrie utilisées sont simulées. Cette approche a permis de valider l'architecture globale de la solution, les flux de communication MQTT, le stockage des données et le fonctionnement du module d'analyse. Elle constitue ainsi une base expérimentale pour une future intégration dans un environnement réel.

Les perspectives de ce projet concernent principalement la connexion de la plateforme à des capteurs et à des machines réelles, son déploiement dans un FabLab opérationnel, ainsi que l'exploitation de données réelles pour améliorer et valider les modèles Data Science. Ces évolutions permettraient de renforcer les capacités de supervision intelligente et de faire évoluer la plateforme vers une solution de jumeau numérique plus complète, adaptée aux environnements de fabrication modernes.

BIBLIOGRAPHIE ET WEBOGRAPHIE

- [1] Neil Gershenfeld, *Fab: The Coming Revolution on Your Desktop*, Basic Books, 2005.
- [2] Klaus Schwab, *The Fourth Industrial Revolution*, World Economic Forum, 2016.
- [3] Samuel Greengard, *The Internet of Things*, MIT Press, 2015.
- [4] Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media, 2019.
- [5] Vercel, *Documentation officielle de Next.js*, <https://nextjs.org/docs>, consultée le 22 mai 2026.
- [6] FastAPI, *Documentation officielle de FastAPI*, <https://fastapi.tiangolo.com/>, consultée le 22 mai 2026.
- [7] Three.js, *Documentation officielle de Three.js*, <https://threejs.org/docs/>, consultée le 22 mai 2026.
- [8] MQTT.org, *Documentation et ressources officielles du protocole MQTT*, <https://mqtt.org/>, consultée le 22 mai 2026.
- [9] SQLite, *Documentation officielle de SQLite*, <https://www.sqlite.org/docs.html>, consultée le 22 mai 2026.
- [10] Auth0, *Introduction aux JSON Web Tokens*, <https://jwt.io/introduction>, consultée le 22 mai 2026.
- [11] Scikit-learn, *Documentation officielle de Scikit-learn*, <https://scikit-learn.org/stable/documentation.html>, consultée le 22 mai 2026.
- [12] Tailwind Labs, *Documentation officielle de Tailwind CSS*, <https://tailwindcss.com/docs>, consultée le 22 mai 2026.
- [13] Blender Foundation, *Documentation officielle de Blender*, <https://docs.blender.org/>, consultée le 22 mai 2026.